

TECHNICAL REFERENCE · LIP 0.3.0

Liquid Interfaces

A protocol for agent systems where the interface is an event, not an artifact

Integration has always assumed that System A must know, in advance and to the byte, what System B expects. Autonomous agents break that assumption from both ends. Liquid Interfaces removes form as a precondition for interaction, and replaces it with intent, negotiation, and dissolution.

PROTOCOL

LIP 0.3.0

GOVERNANCE MODEL

IBAC — Intent-Based Access Control

PAPER

arXiv:2601.21993

REFERENCE IMPLEMENTATION

agentic-bus 0.3.0

SPECIFICATION

github.com/draiven-io/liquid-interfaces

LICENCE

Apache-2.0 / MIT

§ Contents

- 1 The problem it solves
Why static contracts fail against probabilistic callers
.....
- 2 The protocol
Envelope, ten performatives, lifecycle
.....
- 3 Dissolution and the anti-debt principle
The part that is genuinely unusual
.....
- 4 IBAC — Intent-Based Access Control
Authorising the why, not the endpoint
.....
- 5 The authority chain
Identity, tenant, vocabulary, grant, enforcement
.....
- 6 agentic-bus
The reference implementation
.....
- 7 Evidence
What has actually been demonstrated
.....
- 8 Where LIP sits
MCP, A2A, ACP — layering rather than competing
.....
- 9 Objections, answered
The hard questions, with honest answers
.....
- 10 Current state
Shipped, open, and known gaps
.....

HOW TO USE THIS DOCUMENT

Sections 1–4 are the argument. Sections 5–7 are the evidence. Section 8 is what to say when someone pushes back, including the objections that are *correct*. Every number in this document was read from the source at the time of writing rather than recalled.

01 The problem it solves

The history of integration is the history of imposing predictability. RPC, SOAP, REST, GraphQL — different syntax, one premise: the caller must know the callee's shape in advance, exactly.

That premise held because callers were deterministic programs written by people who could read documentation. It stops holding when the caller is an autonomous agent, because the two sides now disagree about what an interaction *is*.

THE AGENT	THE API
Operates probabilistically	Operates deterministically
Interprets natural language	Requires rigid syntax
Adapts to ambiguous context	Demands a fixed schema
Pursues goals — “ <i>reduce logistics cost</i> ”	Executes functions — <code>POST /orders/v1</code>

The industry's answer has been to translate: hand-written connectors, tool descriptions, function-calling schemas. That works, and it produces a specific kind of debt. Every integration is a permanent artefact someone maintains. Every schema change is a breaking change somewhere. The cost does not scale with value delivered; it scales with the *number of pairs* of things that talk.

The observation LIP starts from

When two agents collaborate, the interface between them exists for the duration of one interaction and then has no further purpose. Persisting it — as a connector, a schema, a client library — is not free. It is the thing you will migrate later.

THE CORE CLAIM

If an interface only needs to exist while an interaction is happening, then **making it durable is a cost with no corresponding benefit**. Liquid Interfaces makes the interface ephemeral by construction: it is negotiated when needed and dissolved when finished, leaving nothing to maintain.

This is not an argument that types are bad or that validation is optional. It is an argument about *when* the contract is established. LIP establishes it at interaction time, from what the participants declare about themselves, rather than at design time from what someone anticipated.

02 The protocol

LIP is a message protocol. Every message is a JSON envelope with a performative — an act that changes the state of an interaction. The set of performatives is closed.

2.1 The envelope

Identical for every message type. The payload varies; the frame does not.

```
{
  "protocol_version": "0.3.0",
  "message_id": "0d6f2f1e-1b4a-4c62-9f6a-2c0f0c9a1d33",
  "session_id": "9d1f7c02-...", // the interaction context
  "message_type": "offer", // the performative
  "timestamp": "2026-08-22T14:48:55+00:00",
  "sender": {
    "kind": "agent", // requester | coordinator | agent
    "id": "route-optimizer-01",
    "oidc_subject": "..."
  },
  "trace": { "trace_id": "...", "span_id": "..." },
  "payload": { /* shape determined by message_type */ }
}
```

WHY SENDER.ID IS NOT IDENTITY

`sender.id` is written by the sender. It is a *claim*. A coordinator that authorises on it has authorised whatever the caller typed. Section 4.4 covers what LIP does instead.

2.2 The ten performatives

Closed by design. Adding one changes what the protocol means, which is what the RFC process exists for. In a typed implementation this becomes a finite union rather than a string, and the compiler enforces it.

PERFORMATIVE	DIRECTION	EFFECT
<code>register</code>	Agent → Coordinator	Declares the agent and the capabilities it offers
<code>registered</code>	Coordinator → Agent	Admits or refuses that declaration, with a reason
<code>intent</code>	Requester → Coordinator Coordinator → Agent	Articulates an objective; relayed to candidate agents
<code>offer</code>	Agent → Coordinator	Declares a capability relevant to the intent
<code>accept</code>	Coordinator → Agent	Signals negotiated agreement
<code>reject</code>	Coordinator → Agent	Structured refusal, with cause
<code>execute</code>	Coordinator → Agent	Authorises work under negotiated terms
<code>complete</code>	Agent → Coordinator	Terminates execution, successfully or not
<code>dissolve</code>	Coordinator → All	Invalidates the context; mandatory cleanup
<code>event</code>	Any → Any	Informational progress; changes no state

2.3 The lifecycle

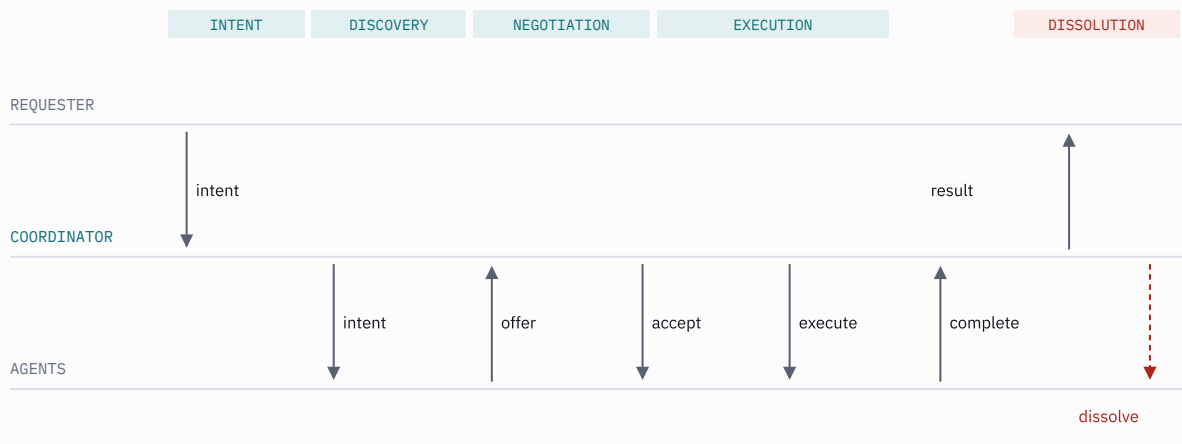


Figure 1. One interaction, end to end. The requester never names an agent, and no agent names another. Everything is mediated, which is what makes the interaction auditable and the participants substitutable.

2.4 Registration is per connection

An agent registers on *every* connection, including reconnections. The coordinator holds its capability registry against the live channel, so an agent that reconnects without registering is connected and undiscoverable — healthy to a process supervisor, absent from the bus.

This is the failure mode RFC 0001 was written to eliminate. Before it, agents registered by sending `complete` with the sentinel `session_id="__registration__"`; registration could not fail visibly, and an agent refused admission stayed connected believing it was live.

2.5 Versioning

Every message carries `protocol_version`. A message without one is treated as `0.1.0` — the last version predating the field — rather than as current. Compatibility is **not symmetric**, and saying so plainly matters more than the version number:

AGENT	COORDINATOR	RESULT
0.2.0	0.2.0	Full handshake
0.1.0	0.2.0	Works — the deprecated form is still accepted
0.2.0	0.1.0	<code>Agent never registers</code> — the coordinator ignores <code>register</code>
0.3.0	0.2.0	Works — the agent is told nothing about grants, and no artifact is validated
0.2.0	0.3.0	Works — declared scopes are read as requests

Coordinators are upgraded before agents. This is the usual ordering for a client/server protocol, but it is a real operational constraint rather than a formality.

03 Dissolution and the anti-debt principle

This is the part of LIP that is genuinely unusual, and the part most worth defending, because it is where the protocol makes a claim no other agent protocol makes.

A conventional contract persists. That is its purpose: it is a promise about the future. The cost is that it must be maintained, versioned, deprecated and migrated — and it constrains interactions nobody has had yet.

A LIP contract is **designed to dissolve**. It has limited temporal validity, exists only inside a session, and is invalidated by `dissolve`, which triggers mandatory cleanup across every participant.

INVARIANT II

Nothing outlives the interaction that created it. When a session dissolves, the negotiated contract, the execution plan, the session memory and the agent bindings are all discarded. What remains is the audit record of what happened — deliberately not the machinery that made it happen.

3.1 What this buys

- **No connector inventory.** Adding an agent means registering capabilities, not editing another agent.
- **No breaking changes between agents.** There is no persistent contract to break; the next negotiation reflects the current declarations.
- **Bounded blast radius.** An interaction that goes wrong is dissolved. It cannot leave a half-configured integration behind.

3.2 What it costs, honestly

- **Every interaction pays negotiation cost.** Discovery and planning are real work, and for a hot path called ten thousand times a second they are the wrong trade. LIP is for coordination, not for the inner loop.
- **Reproducibility takes deliberate effort.** If the plan is composed at runtime, "why did it do that" is answerable only if you recorded it. Hence the audit trail is not optional in the reference implementation.

3.3 Dissolution during execution

An agent may have work in flight when `dissolve` arrives. The specification requires it to stop, and to **not** send `complete` afterwards: the interaction context the completion would report against no longer exists. Two conformant implementations behaving differently here is precisely the ambiguity a specification exists to prevent, so it is stated explicitly.

04 IBAC – Intent-Based Access Control

The strongest independent criticism of agent protocols is not that they are slow or immature. It is that they cannot express *purpose*, *delegation*, *revocation* or *provenance* — so authorisation stays at the endpoint while the decision that mattered was made somewhere else. IBAC is the answer to that criticism.

4.1 Why RBAC and ABAC do not fit

MODEL	AUTHORISES	WHY IT IS INSUFFICIENT HERE
RBAC	A role calling an endpoint	An agent's role is stable; its <i>purpose</i> changes every request. The same agent legitimately reading customer records for a support query is exfiltrating when the purpose is different.
ABAC	Attributes of subject, object, environment	Closer — but the attribute that decides this case is <i>why</i> , and “why” is not an attribute of the subject or the object. It is a property of the interaction.
IBAC	The intent behind the request, at every stage of it	Evaluates the interaction rather than the call. The same endpoint, the same identity, the same data — different answers, because the purpose differs.

4.2 Five evaluation points

A request is not judged once. It is judged five times, each against what is known by then — because what is knowable changes as the interaction proceeds.



Figure 2. The same request, judged five times. Denial at point 1 is cheap and total; denial at point 5 is the last defence before data crosses a boundary.

4.3 Three layers, ANDed

Each evaluation runs three layers, and the **strictest decision wins**. The layers are combined, never short-circuited: any layer must be able to block what another approved, and that guarantee is worth nothing if one `ALLOW` ends the evaluation.

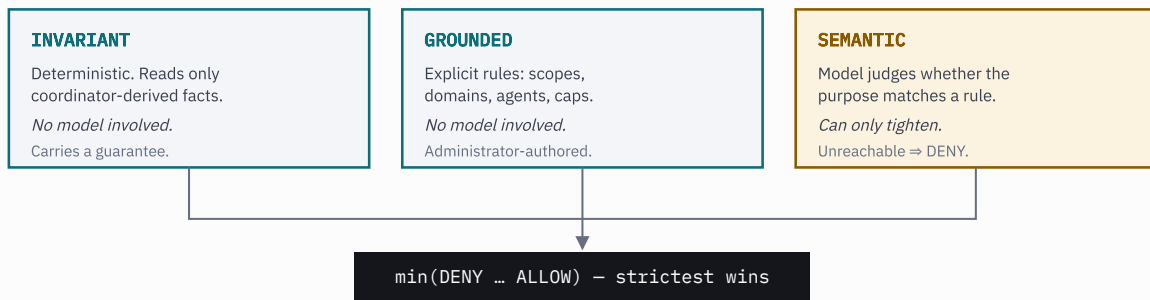


Figure 3. Because the combiner takes the strictest outcome, **the model layer can only ever tighten a decision, never loosen one.** An LLM in this path cannot grant access that the deterministic layers withheld.

That property is worth stating precisely, because it is the answer to the most serious objection to putting a language model anywhere near authorisation. The precedence order is `DENY` → `REQUIRE_HUMAN_APPROVAL` → `ALLOW_WITH_SCOPE` → `ALLOW`, and the combiner selects the minimum. A compromised or confused model can refuse things it should have permitted. It cannot permit things it should have refused.

4.4 Declared versus derived

Every fact entering an IBAC evaluation is classified, and the classification decides which rules can carry a guarantee.

DECLARED — THE ACTOR WROTE IT

`intent_text`, `purpose`, `context`,
`claimed_agent_id`

Can be reworded until it passes. A rule reading only declared input is a heuristic, however well written.

DERIVED — THE COORDINATOR ESTABLISHED IT

`authenticated_subject`,
`authenticated_agent_id`, `identity_verified`,
`session_id`, `evaluation_point`

Resolved from the authenticated connection and the coordinator's own state. Cannot be argued away by phrasing the request differently.

WHY THIS DISTINCTION IS LOAD-BEARING

The invariant layer reads **only** derived facts. That is what lets it carry a guarantee. A result is marked `relies_on_declared_input` when the deciding layer read something the actor asserted — so an operator can tell which of their controls are boundaries and which are merely good defaults.

The concrete attack this closes: an agent putting a colleague's identifier in `sender.id`. The manifest compares the claim against the identity resolved from the connection, and impersonation becomes a detectable, attributable event rather than a successful one.

4.5 Fail-closed

POLICY

An evaluation that did not happen is not permission. If the semantic layer cannot reach its model, the decision is `DENY` — not “fall back to the deterministic rules and continue”.

This is a deliberate posture with a real cost: a model outage stops work that would otherwise have been permitted. It is chosen because the alternative is worse, and because the alternative is what systems drift into by default. Fail-open authorisation is rarely designed; it accumulates, one exception handler at a time, and it is invisible until the day it matters.

4.6 Bounded capabilities

An approval that grants nothing specific has approved everything. When IBAC allows, it issues a **capability** — a bounded, expiring grant naming exactly what may be done.

PROPERTY	MEANING
<code>principal</code>	Who holds it — the authenticated subject, not a claimed id
<code>scopes</code>	Exactly what it permits; anything outside is denied
<code>expires_at</code>	Default TTL 3600s. An expired capability is refused on presentation
<code>session</code>	Bound to the interaction, and destroyed when it dissolves

The sequence is: semantic authorisation decides *whether*; a capability records *what*; deterministic execution enforces it. The judgement is made once, by the layer able to reason about purpose, and everything downstream is a mechanical check.

05 The authority chain

IBAC decides whether a purpose is permitted. That leaves a second question it does not answer — *with whose authority, and bounded by what?* — and 0.3.0 is largely the answer to it.

Before 0.3.0 the honest description of this layer was: an agent declared the permissions it wanted, and the coordinator wrote them down. Nothing verified the connection, nothing bounded the agent by the requester it acted for, and a permission name was a free string nobody had a list of — so `carrier:route` registered exactly as successfully as `carrier:quote`, and every rule guarding that capability silently stopped applying.

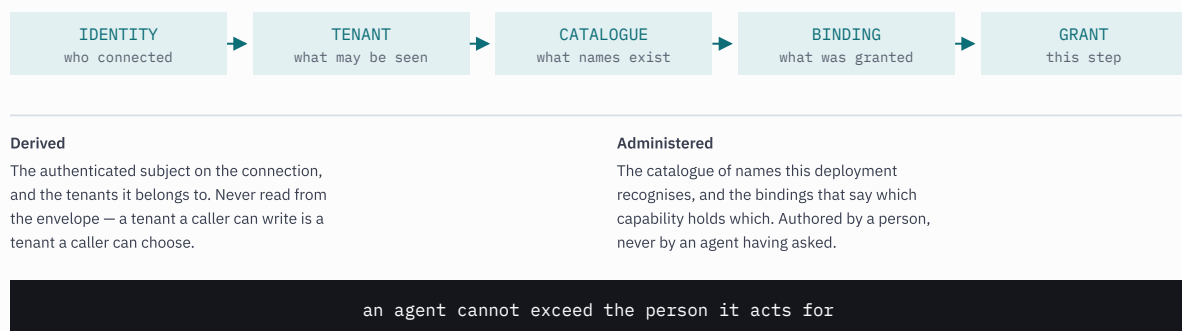


Figure 4. Five narrowings. The first two are facts about the connection; the next two are decisions a person made; the last is their intersection for one step.

5.1 Identity — who is connecting

LIP §12 requires agents to be authenticated before participating. Until 0.3.0 the reference implementation did not do it, in a way worth describing precisely because the shape recurs: a verifier was constructed and never called, an `auth_handler` parameter existed and was never passed, and the identity map was read in two places and assigned in none. Every part looked present.

A credential now rides on the WebSocket upgrade and is verified against the configured issuer. What it proves is deliberately narrow — **who is connecting**, not what they may do.

AUTHENTICATION IS NOT ENTITLEMENT

An agent that authenticates correctly and then registers under a colleague's `agent_id` has authenticated perfectly and impersonated anyway. So an `agent_id` binds to the first authenticated subject that claims it, and afterwards only that subject may. A connection can never overwrite a binding — otherwise the binding is self-issued, and worth nothing.

5.2 Tenant – what may be seen at all

The data model carried tenants from the beginning and the coordination path never consulted them. Discovery did this:

```
summaries = registry.capability_summaries() # every agent, every tenant
result = await chain.ainvoke({"capabilities": str(summaries), ...})
```

Two failures, and the second is the one that gets overlooked. The model could *pick* another tenant's agent — and it *saw* them, which is disclosure whatever it then picked. A capability description names things: “*query ACME Corp's payroll database*” is a customer's name in another customer's prompt.

So the filter applies to discovery's **input**, before the summaries are assembled. Filtering the candidates afterwards would be too late.

ASSIGNMENT	VISIBILITY
Agent assigned to no tenant	Global — visible to everyone
Agent assigned to tenants	Visible only to requesters sharing one
Requester resolving to no tenant	Global agents only

Tenancy switches on by **assigning agents**, not by setting a flag. A bus that never heard of tenants keeps working; one that uses them is isolated. The third row means an unidentified caller in a fully-assigned deployment sees nothing — the intended answer rather than an edge case.

5.3 Vocabulary – what the names mean

A scope name belongs to a **deployment**, not to an agent. Requiring an agent to declare `payments:write` requires its author to know that *this* organisation calls it that, so an agent built for one bus needs editing for the next — the per-pair coupling §3 exists to remove.

An agent therefore declares what it *does*; a coordinator decides what that requires:

```
AgentCapability(
    capability_id="carrier_rate_quote",
    description="Obtains and compares freight quotes from multiple carriers",
)
```

`required_scopes` remains on the wire and its meaning changed: a **request**, recorded and surfaced to an administrator, never an authority.

REFUSAL IS HOW THE VOCABULARY PROPAGATES

Vocabulary a coordinator does not recognise is refused *with the catalogue*. That answers the obvious question — *how does an agent discover which names exist?* — without a discovery endpoint the agent would have to be built against. An implementer learns by being corrected, at the moment they err, instead of from documentation that goes stale.

It also converts the failure that motivated all of this. A misspelt scope used to register successfully into a protection gap; it now surfaces as a request, next to the correct spelling.

Matching is hierarchical: `payments:*` covers `payments:write` ;
`payments:write` covers only itself, because a grant is not widened by having children.

5.4 Grant – the two empties

Narrowing happens twice with opposite meanings for an empty bound, and the distinction is the single easiest thing to get wrong here.

BOUND	EMPTY MEANS	BECAUSE
Requester's credential	no limit expressed	A missing <code>scope</code> claim usually means the identity provider does not use scopes, not that its holder may do nothing. Reading absence as denial would refuse every caller whose IdP is configured differently, to protect nothing.
Administrator's binding	nothing granted	The absence of a decision is not permission to proceed. An unbound capability holds nothing, deliberately.

They are separate functions rather than a parameter, so every call site must say which meaning it wants. Using the wrong one is not a subtle bug — it turns an unbound capability into an unrestricted one.

5.5 Enforcement – and which attacker it stops

A granted scope used to reach the agent and be ignored. Now work that a permission governs says so where the work happens:

```
from agentic_bus import require_scope

@tool
def issue_refund(order_id: str) -> str:
    require_scope("payments:refund")
    ...
```

A coordinator cannot police what runs inside an agent's process. That is where the trust boundary is, not a shortcoming to fix — so it is worth being exact about what this defends, because overstating it would be worse than not having it.

NOT A MALICIOUS AGENT BINARY

Code that controls the process can call anything and report anything. Nothing running in that process could stop it, and an attacker who controls an agent's code has already replaced everything this could have protected.

YES A COMPROMISED AGENT BRAIN

Which is the realistic threat when a language model drives the agent. An injected prompt can persuade the model to call `send_email` ; it cannot persuade the invocation path to skip the guard, because the guard is not in the model's control. **The model chooses what to call. It does not choose whether the call is checked.**

5.6 Reconciliation – detection, with attribution

The completion reports `used_scopes` and `denied_scopes` , recorded by the invocation path rather than volunteered by the model, and the coordinator compares them against what it granted. Exceeding the grant is recorded as `scope.exceeded` , at critical severity, against the agent — so a pattern is visible across sessions rather than only within one.

A refused attempt is recorded too, and is the more interesting half: an agent that *tried* to refund and was stopped is a finding, where one that succeeded is a Tuesday.

WHY DETECTION IS WORTH BUILDING

This is detection, not prevention, and the difference is the point rather than an excuse.

Attribution is what makes revocation mean something. A bus that can name the agent, the session and the scope can revoke that agent and know what it was doing; one that cannot has a revocation button that changes nothing it can describe.

5.7 What an offer promises

Every offer in this protocol carries an `output_schema` , and until 0.3.0 nothing had ever read one. An agent could promise `{"routes": [...]}` , deliver `{"result": "ok"}` , and the interaction would proceed — until the next step consumed the artifact and assumed a field that was not there. That is the expensive shape: the failure surfaces one or two hops from its cause, and debugging starts at the wrong agent.

Three outcomes are kept distinct, because collapsing them makes an audit trail claim more than it knows:

OUTCOME	MEANING
Checked and matched	The artifact is the shape its offer declared
Checked and violated	<code>schema_violation</code> — named agent, named promise
Nothing was promised	No schema declared. <i>Not a pass</i> — nothing was verified

An absent schema is explicitly not a failure. An agent that cannot describe its output is still useful, and demanding a schema from every agent would exclude the exploratory ones this protocol exists to accommodate. **Silence is honest; a promise that is broken is not.**

5.8 Postures

Every guarantee above arrives switched off, and switches on by configuration rather than by a flag nobody sets. The reason is migration: turning enforcement on in a bus that has been running refuses work that used to pass, and an operator should be able to see that before it happens rather than during.

SETTING	OFF (DEFAULT)
<code>AGBUS_OIDC_ISSUER</code>	Credentials are parsed but not cryptographically verified
<code>AGBUS_REQUIRE_AGENT_AUTH</code>	A peer without a credential is admitted, carrying <i>no</i> identity
<code>AGBUS_SCOPE_CATALOGUE_ENFORCED</code>	Unrecognised names are catalogued on sight, and warned about
<code>AGBUS_SCOPE_ENFORCED</code>	An ungranted step logs loudly and runs
<code>AGBUS_ARTIFACT_VALIDATION_ENFORCED</code>	A violation is recorded; the step stands

Setting an issuer implies the others' intent and cannot be loosened back: a deployment that configured an identity provider did not mean *sometimes*.

06 agentic-bus

The reference implementation: a coordinator, an agent SDK, an admin API and a dashboard. Python 3.11+, Apache-licensed, published to PyPI.

23.6k

lines of implementation

11.4k

lines of tests

741

tests passing

5.1 What the coordinator does

1. **Admission** — authenticates the connection, checks the agent is neither rejected nor revoked, and verifies the credential is entitled to the claimed `agent_id`.
2. **Intent processing** — admits or refuses the intent (IBAC point 1), then decomposes it into sub-intents.
3. **Discovery** — semantic matching against the capability registry.
4. **Negotiation** — collects offers, evaluates eligibility (point 2), composes a plan (point 3).
5. **Execution** — authorises each step (point 4), dispatches, supervises, scores.
6. **Validation** — optionally has a nominated agent validate the result, with bounded renegotiation.
7. **Dissolution** — invalidates the context and archives the audit record.

5.2 Transport independence

The coordination logic never touches the wire — it asks a transport for a peer and sends an envelope. Two transports ship:

TRANSPORT	USE
<code>WSServer</code>	Agents connect over WebSocket from anywhere — separate processes, containers, machines, languages.
<code>LocalTransport</code>	Agents attach in-process. No port, no socket, no serialisation. Lets the coordinator run as a <i>library</i> inside an application that already owns a process.

Envelopes are deep-copied across the in-process boundary. A socket hands each side its own object for free; skipping the copy would let code pass locally and misbehave over a network, which is worse than having no local transport at all.

5.3 Agent-side ergonomics

```
# Python
class Weather(BaseAgent):
    def capabilities(self):
        return [AgentCapability(capability_id="forecast",
                                required_scopes=["weather:read"])]

    async def execute_task(self, payload, context):
        return {"forecast": await lookup(payload["city"])}

await Weather(agent_id="weather-01").run_forever()
```

That is the entire surface. There is no router to configure and no other agent to point at: the coordinator discovers `forecast` from the registration and addresses it when an intent calls for it. The SDK handles registration per connection, jittered reconnection, per-session cancellation, and refusing to execute before `execute` arrives.

5.4 Security posture

CONTROL	BEHAVIOUR
Agent authentication	Bearer credential on the WebSocket upgrade, verified against the configured issuer. See §5.1.
Tenant isolation	Applied to discovery's input, so another tenant's capability descriptions never enter a prompt. See §5.2.
Scope enforcement	Catalogued vocabulary, administrator-authored bindings, checked at the point of use, reconciled afterwards. See §5.3–5.6.
Development mode	Admitted without a credential but carrying <i>no</i> identity, so <code>identity_verified</code> is false everywhere it matters.
Entitlement	An <code>agent_id</code> binds to the first authenticated subject that registers it. Only that subject may claim it afterwards; a connection can never overwrite a binding.
Revocation	Enforced in ephemeral mode too — revocation escapable by reconnecting differently is a suggestion.
Audit	Every coordinator decision is recorded with actor, target, severity and reason.

07 Evidence

A specification is a claim. This section is what has actually been demonstrated, and it is the material worth leading with in a technical argument.

6.1 An independent implementation, from the document alone

A TypeScript agent client was written from the specification and the published JSON Schemas, **without reference to the Python implementation**. That constraint is the point: anything the document leaves out shows up as a guess.

Result: **fully conformant, zero advisory warnings**, in roughly 300 lines over one runtime dependency. The exercise also surfaced four gaps, three of which were genuine defects now fixed in the specification:

1. **How a credential is carried was undefined.** §12 required authentication and the envelope had `oidc_subject`, but nothing said how a token reaches the coordinator. Now specified as `Authorization: Bearer` on the upgrade request.
2. **That the coordinator relays `intent` to agents was only inferable** from the fact that an agent could not otherwise know what to offer against. Now stated.
3. **Dissolution mid-execution was undefined** — whether to still send `complete`. Two conformant implementations could have behaved differently. Now stated.
4. **An ordering trap in the phrasing.** “An agent joins by sending `register` and waiting for `registered`” reads as *send, then wait* — which drops the acknowledgement that arrives in between. The specification now says an agent must be receiving before it registers.

6.2 A conformance suite

Twelve graded requirements, protocol-only, so it does not care what language an agent is written in. MUST failures break conformance; SHOULD failures warn.

REQUIREMENT	LEVEL	CHECKS
LIP-REG-001	MUST	<code>register</code> is the first message on a connection
LIP-REG-002	MUST	The register payload carries an <code>agent_id</code> and validates
LIP-REG-003	SHOULD	At least one capability is declared
LIP-INT-001	MUST	A matching <code>intent</code> produces an <code>offer</code>
LIP-EXE-001	MUST	No execution before <code>execute</code> arrives
LIP-EXE-002	MUST	<code>complete</code> is emitted when execution ends
LIP-DIS-001	MUST	Work stops on <code>dissolve</code>
LIP-MSG-001	MUST	Every message is a valid envelope
LIP-MSG-002	MUST	Each payload matches the schema for its type
LIP-MSG-003	MUST	Only defined performatives are used
LIP-VER-001	MUST	Every message carries a <code>protocol_version</code>
LIP-ACK-001	SHOULD	Serving continues when <code>registered</code> never arrives

WHY THIS MATTERS MORE THAN THE PASS

The suite is also run against *deliberately broken* agents, each violating exactly the requirement it covers. A conformance suite everything passes certifies nothing.

6.3 A working end-to-end run

One sentence — “*a storm has closed the Port of Rotterdam; reroute our refrigerated pharmaceuticals from Hamburg to Madrid within five days, and confirm cold-chain warehouse space*” — decomposed into sub-intents, discovered four agents that were never wired to each other, collected and evaluated their offers, composed a six-step plan, paused at a human approval gate, executed, validated, scored each agent, and dissolved. Elapsed: **47 seconds**.

No agent in that run knew about any other. Adding a fifth would have required registering it, and editing nothing.

7.4 What checking it actually found

Worth listing, because a protocol's credibility rests less on what it claims than on what happened when someone checked. Each of these was found by building against the specification, and each is now closed:

- A specification gap in **how a credential is carried**, found by an independent implementation having to guess.

- Agents were **never authenticated** on the bus path — a verifier constructed and never called, an identity map read and never written.
- Registration performed **no admission check**, so a revoked agent could return by reconnecting.
- The execution guard's scope comparison **iterated an empty list**, because its only call site omitted the argument.
- A misspelt scope **registered successfully**, into a protection gap.
- Discovery described **every tenant's agents** to a language model.

Two were found by a conformance run, one by a test asserting a refusal rather than a success. That is the argument for both: a suite everything passes certifies nothing, and a check that looks present is not a check that runs.

08 Where LIP sits

The most common misreading is that LIP competes with MCP or A2A. It does not, and arguing that it does is the fastest way to lose a technical audience.

PROTOCOL	ANSWERS	RELATIONSHIP TO LIP
MCP	How does a model call a tool?	Complementary and below. An MCP server is a capability an agent can offer. agentic-bus ships an MCP bridge for exactly this.
A2A	How do two agents exchange messages?	Complementary and below. A2A is transport and framing between peers; LIP is what the messages <i>mean</i> and who decides.
ACP	How are agents described and discovered?	Overlapping at discovery. LIP adds negotiation, authorisation and dissolution around it.
LIP	Who should do this, may they, under what terms, and what happens afterwards?	—

THE POSITION TO ARGUE

MCP won tools. A2A is winning messaging. Neither expresses **purpose, delegation, revocation or provenance** — and independent analysis of the agent-protocol landscape identifies exactly that gap. LIP is a governance and coordination layer that sits *above* them, not a replacement for either. The right posture is “we layer on A2A”, not “we compete with A2A”.

09 Objections, answered

Including the ones that are correct. A defence that has no concessions in it is not credible to the audience you most want to convince.

“This is a language model deciding access control. That is disqualifying.”

It would be, if the model could grant. It cannot. Decisions from three layers are combined by taking the **strictest**, and the two deterministic layers run regardless — one of which reads only coordinator-derived facts and never invokes a model at all. The semantic layer can turn `ALLOW` into `DENY` ; the ordering makes the reverse impossible.

And when the model is unreachable, the answer is `DENY` , not “continue without it”.

“Enforcement inside the agent is enforcement the agent can skip.”

Correct, and stated as such in the code rather than defended. A coordinator cannot police what runs inside a process it does not control — that is where the trust boundary is, and no design moves it.

What the distinction buys is worth being precise about. An attacker who controls an agent's *code* has already replaced everything the guard could have protected. An attacker who controls its *prompt* — the realistic case when a model drives the agent — has not: the model chooses what to call, and does not choose whether the call is checked or recorded. The completion's usage report therefore reflects what happened rather than what the agent would like to say happened.

Downstream, that is detection with attribution rather than prevention, and attribution is what makes revocation mean something.

“You have replaced API schemas with a permission catalogue. The contract just moved.”

Yes — and pretending otherwise would not survive a first reading. A deployment maintains a scope catalogue and a set of bindings, and those are artefacts someone owns.

The difference that matters is what they scale with. A schema inventory grows with the number of *pairs* of systems that talk; a permission catalogue grows with the deployment's own authorisation model, which is a list most organisations already keep in an identity provider. And it is reviewed once, at admission, rather than every time the other side changes shape.

Any defence of this protocol that claims contracts have disappeared is overselling it, and a senior engineer will find the seam in under a minute.

“Without a fixed contract, what stops the system hallucinating one?”

Two things, and both were true until 0.3.0.

An artifact could be any shape. Every offer carries an `output_schema` and nothing read one, so a promise and a delivery could differ freely. That is now checked, with three outcomes kept distinct — including *nothing was promised*, which is not a pass (§5.7).

A plan could name a capability nobody offered — in a coordinator that composes with a model. This is a correction I had to make to my own proposal: this one does not.

`compose_offers` builds each step by reading fields off an accepted offer, so its steps reference real capabilities by construction, and the only model in that path *explains* the plan rather than producing it. The requirement in RFC 0002 stands as a constraint on coordinators that take the freedom the specification allows.

The general answer is the same in both cases: **self-declared vocabulary is not a fact until it has been admitted**. RFC 0002 applies that to what an agent produces; RFC 0003 to what it asks for.

“Negotiating every interaction is far too slow.”

Correct, for the wrong workload. LIP is a coordination protocol, not a data path. Discovery, planning and semantic authorisation cost seconds, and no argument makes that acceptable in a hot loop called thousands of times a second.

It is the right trade when the alternative is a human deciding, or a connector being written. Use it where the work is coarse-grained and the coordination is the hard part — and call the deterministic path directly where it is not.

“If plans are composed at runtime, debugging is impossible.”

Only if nothing is recorded. Every decision is auditable: the intent, its decomposition and rationale, which agents were discovered and why, each offer, every IBAC evaluation with the layer that decided and its reason, each execution step with latency and score, and the dissolution. A completed session is reconstructible from its archive.

The honest form of this objection is different and worth conceding: *reproducibility* is harder than in a static system, because the same intent may produce a different plan tomorrow. Determinism was traded for adaptability deliberately.

“This is a workflow engine with extra steps.”

The distinguishing test is what happens when you add a participant. In a workflow engine, adding a step means editing the workflow — the topology is authored. Under LIP, an agent registers its capabilities and becomes discoverable; nothing else changes, and no existing agent is modified.

The second test is what happens when a step fails. A workflow retries or fails the run. A coordinator can renegotiate — bounded, three rounds — because the plan was never the authority; the intent was.

“Who maintains this? It looks like one company's internal protocol.”

Fair, and currently accurate. What exists to answer it: an open specification with an RFC process, published JSON Schemas with CI enforcing that implementation and schema cannot drift, a language-agnostic conformance suite, and two independent client implementations — one written from the document alone, precisely to prove the document is sufficient.

What does not yet exist is adoption outside its origin. That is the real gap, and it should be conceded rather than argued around.

“Ephemeral interfaces mean losing type safety.”

The performative set is closed and the envelope is schema-validated — in TypeScript `MessageType` is a finite union the compiler enforces, and payloads validate against published JSON Schemas on every message. What is *not* statically typed is the relationship between two agents, because that relationship did not exist until the interaction started.

The trade is real: static typing at the boundary, runtime validation across it.

10 Current state

9.1 Shipped

COMPONENT	VERSION	STATUS
LIP specification	0.3.0	Published; RFCs 0001–0003 accepted
agentic-bus (Python)	0.3.0	Published on PyPI
JSON Schemas	0.3.0	Generated from the models; CI fails on drift
Conformance suite	—	<code>agbus conformance</code> ; 12 graded requirements
TypeScript client	0.2.0	In-repo and conformant; npm release pending
IBAC engine	—	Five points, three layers, fail-closed, bounded capabilities
Admin dashboard	—	Next.js; agents, IBAC, audit, history, negotiation view

9.2 Known gaps

Stating these first is more persuasive than being asked about them. Three that appeared here at 0.2.0 — no capability authority, no tenant isolation on the coordination path, and no enforcement of a granted scope — closed in 0.3.0. The ones below did not.

- **A scope is still a statement of intent.** Grants are catalogued, bound by an administrator and checked at the point of use — but the check lives inside the agent's process. It stops an agent whose model was talked into exceeding its authority; it does not stop one whose *code* is hostile, and nothing running in that process could. What the coordinator gets is detection with attribution, which is what makes revocation mean something.
- **An agent that under-declares is not caught.** Enforcement acts on what an agent said it needs. Declaring less passes.
- **Sequential execution only.** No parallel fan-out with synthesis.
- **No cost or token budget abstraction.** Negotiation and renegotiation are bounded by round count, not spend.
- **Adoption.** One production lineage, no external users yet.

9.3 What would change the argument

The strongest thing that could happen to this protocol is not another feature. It is a second organisation depending on it, because a framework with one user invents requirements while a framework with users discovers

them. Everything in section 7 exists to make that step cheap for whoever takes it.

Specification and RFCs: github.com/draiven-io/liquid-interfaces · Reference implementation: github.com/draiven-io/agentive-bus · liquidinterfaces.org · Paper: [arXiv:2601.21993](https://arxiv.org/abs/2601.21993)

Figures and metrics in this document were read from the source repositories at the time of writing. Where a claim is a proposal rather than shipped behaviour, it is marked as such.